Main memory: what this chapter explains

- Derive base-and-limit, segment, and page translations
- Distinguish internal and external fragmentation
- Calculate TLB costs and explain page-table levels

Central question. How can a logical address space map safely onto finite, physically scattered memory?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Address protection: check before relocation

In a single contiguous-region model, base is the start of a process's physical region and limit is its length. Given CPU offset d, hardware checks `0 ≤ d < limit` before calculating physical address `base + d`; otherwise it raises a protection exception. Privileged software controls base and limit.

For base=`0x8000`, limit=`0x2000`, offset `0x1FFF` maps to `0x9FFF`; `0x2000` equals the length and is rejected. A multi-byte access must have its entire range inside the region, not just its starting address.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Swapping

- Classical swapping moves an entire process image to backing storage to free RAM. Restoration needs the saved image, address-space metadata, and an appropriate placement or relocation mechanism.
- Demand paging selectively loads pages rather than requiring the whole image. Its granularity and cost differ from whole-process swapping.

For an 80 KiB process and 100 KiB available backing store, preserve its required state on swap-out. A DMA buffer must remain valid until I/O completes, for example by pinning its frame.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Contiguous allocation: enough total space can still fail

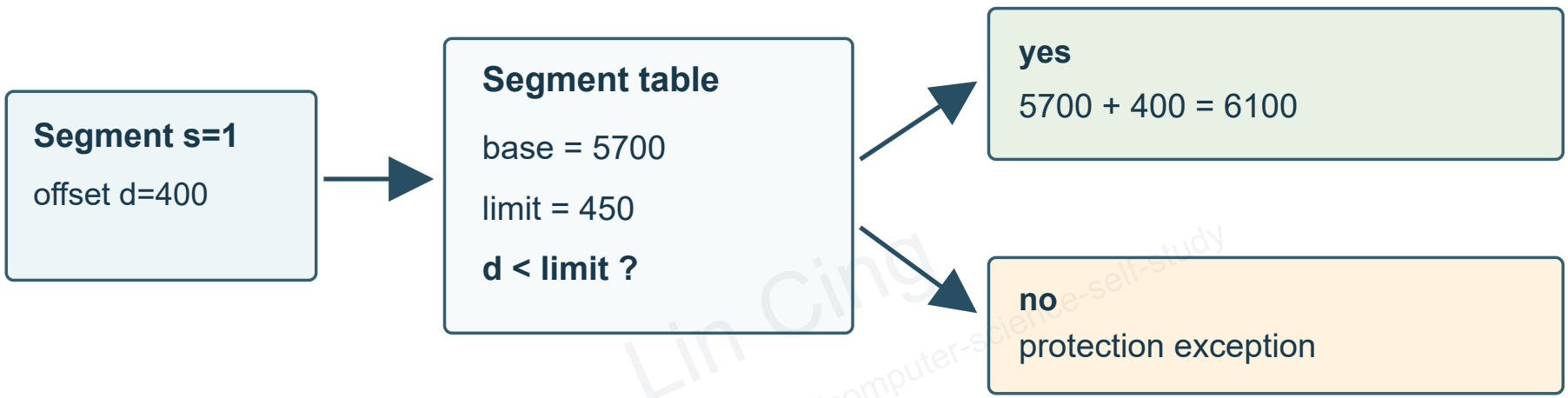
Holes in address order are 100, 500, 200 KiB; a new contiguous request needs 180 KiB. First-fit selects 500 and leaves 320; best-fit selects 200 and leaves 20; worst-fit selects 500 and leaves 320. Each policy is independently applied to the same initial state.

Four separated 5 KiB holes total 20 KiB, yet none accommodates a contiguous 10 KiB request: this is **external fragmentation**. Allocating two 4 KiB units for a 6 KiB request leaves 2 KiB inside the allocated units: **internal fragmentation**. Compaction requires relocation and coordination with active I/O and address references.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Segmentation: each segment has its own limit

Protection check for a segment address



Offset 449 is valid; offset 450 is out of bounds.

Let segment 0 have base=1000 and limit=339, segment 1 base=5700 and limit=450, and segment 2 base=6300 and limit=1280, all in bytes. `<0,330>` is valid because `330<339` and maps to 1330; `<1,400>` maps to 6100; `<2,1380>` raises an exception because `1380≥1280`.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Segmentation: each segment has its own limit (continued)

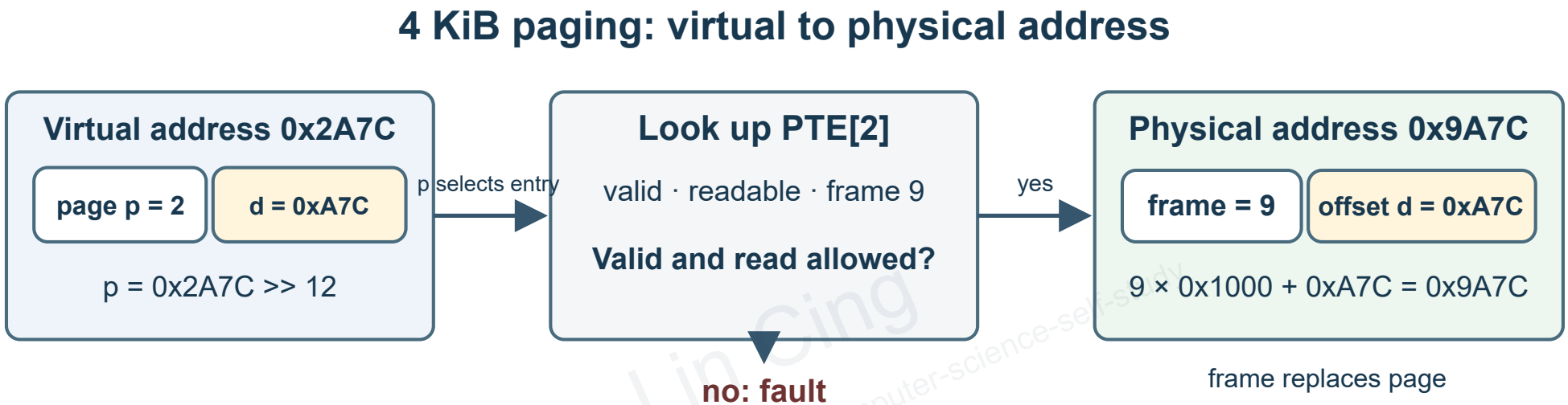
The limit is a length, not the last legal offset: segment 1 permits offset 449 and rejects 450. Segments may have different lengths, permissions and physical bases; each segment occupies a contiguous physical region in this model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Paging

- Split a virtual address into page p and offset d. If an authorized resident entry maps p to frame f, physical address=f*page_size+d. The offset is unchanged.
- Smaller pages reduce tail waste but increase the number of entries for a fixed range. Larger pages increase TLB coverage per entry and may reduce table work, while increasing allocation and transfer granularity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

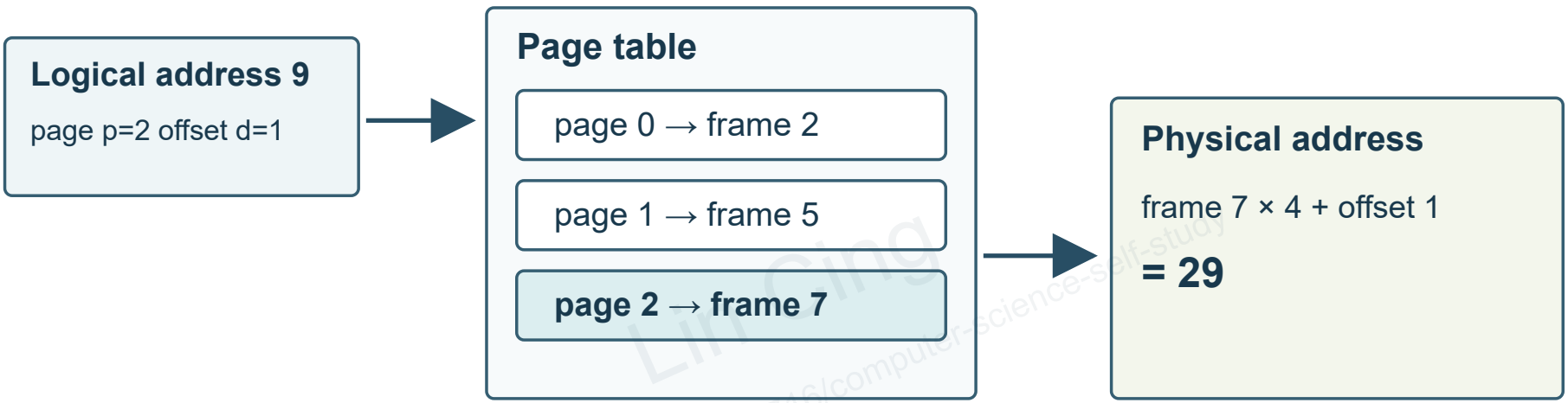


Original vector diagram for this course.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Translate address 9 to address 29

4-byte pages: split, then look up



The offset is unchanged; the table maps a page number to a frame.

Page size is 4 bytes. For logical address 9, `p = floor(9/4) = 2` and `d = 9 mod 4 = 1`. The table maps page 2 to frame `f = 7`; the physical address is `7×4+1 = 29`. The offset is preserved. Page and frame numbers need not match.

Translate address 9 to address 29 (continued)

Only valid pages 0, 1 and 2 are shown. An invalid page-table entry does not specify a frame; the system raises a fault or handles a missing page according to its policy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

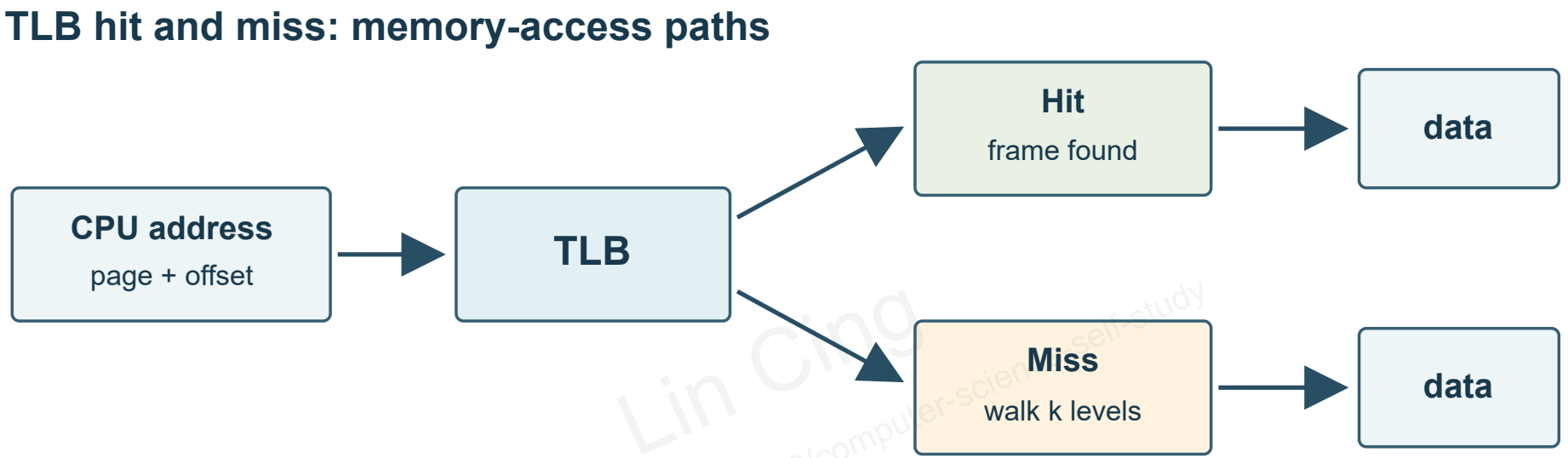
Address bits and fragmentation

With 1 KiB = 2^{10} bytes per page and 8 = 2^3 pages per address space, the logical address has 3 page bits plus 10 offset bits, **13 bits** total. With 32 = 2^5 physical frames, the physical address has 5 frame bits plus the same 10 offset bits, **15 bits** total. Addresses are byte-based and the full encoding ranges are usable.

A contiguous 32 KB logical region in 10 KB allocation units needs four pages, or 40 KB. If the first three are full, the last uses 2 KB, leaving 8 KB of internal fragmentation. Here KB means 1000 bytes for this capacity example; 1 KiB in the binary address example is 1024 bytes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TLB hit and miss timing



Serial model: hit = TLB + data; miss = TLB + k table reads + data.

On a TLB hit, use the frame to access data. On a miss, consult the page table first. Assume serial lookup, equal memory-access costs, fixed walk depth, and no cache, page-walk cache, replacement or page fault.

TLB hit and miss timing (continued)

For one level, TLB lookup = 10 ns, RAM access = 200 ns and hit rate = 0.8. A hit costs `10+200=210 ns`; a miss costs `10+200+200=410 ns`. EAT is `0.8×210 + 0.2×410 = 250 ns`. This is an average under the stated model, not a measured latency for every CPU.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Adding a page-table level

Now use two page-table levels, a 20 ns TLB lookup, 200 ns RAM access and hit rate 0.8; keep the other assumptions. A hit costs `20+200=220 ns`. A miss reads two entries and then data, costing `20+2×200+200=620 ns`.

EAT is `0.8×220 + 0.2×620 = 300 ns`. Both TLB lookup time and walk depth changed between these examples, so the entire 50 ns difference cannot be attributed to the extra level. Real hardware may overlap lookups or cache intermediate entries.

In this serial model, hit rate h, lookup time t, memory time m and k table reads on a miss give EAT = h(t+m)+(1−h)(t+km+m).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Protection and sharing

- Entries may encode read, write, execute, privilege, and presence rules. Different address spaces can share a physical frame through explicitly authorized mappings.
- Read-only code sharing saves frames. An accidentally writable or executable mapping can break isolation. Presence alone does not grant every operation.

For example, P1 and P2 can map the same frame as user-readable and read-only. Both can read the same contents; user writes fail. Removing one mapping leaves the other intact. Before reclaiming the frame, account for every reference and invalidate affected TLB entries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-level tables: storage depends on mappings

Use 32-bit byte addresses, 4 KiB pages and 4-byte entries. A 4 KiB table holds $1024 = 2^{10}$ entries, so the address splits into `10-bit root index | 10-bit leaf index | 12-bit offset`. Each leaf table covers 1024 pages, or 4 MiB of virtual address space.

To map one page, a complete root plus one leaf needs `4 KiB + 4 KiB = 8 KiB` of table storage; this excludes the data page. Mapping all 2^{20} virtual pages needs 1024 leaves: `4 KiB + 1024×4 KiB = 4 MiB + 4 KiB`. Sparse mappings avoid unused leaves; 8 KiB is not a general maximum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inverted tables: count physical frames

For 1 GiB physical memory and 8 KiB pages/frames, there are $2^{30} / 2^{13} = 2^{17}$ frames. If each per-frame record is exactly 4 bytes, these records occupy $2^{17} \times 4 = 2^{19}$ bytes = 512 KiB.

This counts only the frame-indexed records. A hash/index for `(address-space ID, virtual page)`, collision chains and records for multiple mappings of a shared frame can require extra space. A real record must also hold its required identifiers and permissions; 4 bytes is an example assumption, not a universal size.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

IA-32 linear-address walk

Fix IA-32 protected mode with `CR0.PG=1`, `CR4.PAE=0`, ordinary 4 KiB pages, and linear address 0x12345678 after segmentation. The selector first locates an LDT/GDT descriptor; segment base/limit and rights produce the linear address. Paging translates that result, so the given number is not the pre-segmentation logical address.

A 4 KiB directory/table has 1024 four-byte entries, giving 10/10/12 bits. For linear L,

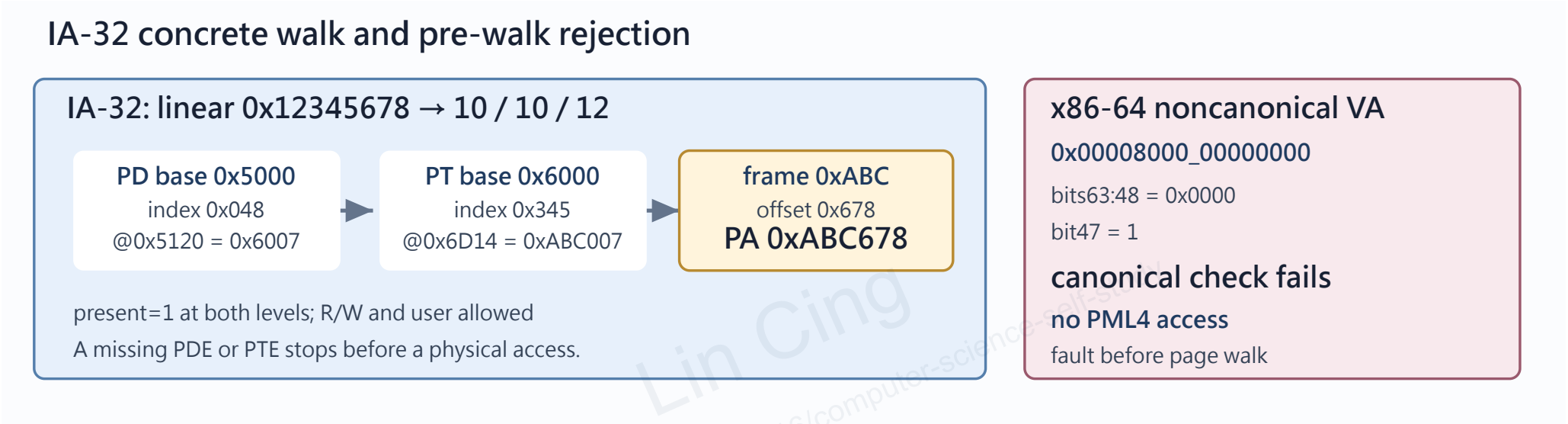
$$PDI = (L \gg 22)\ \&\ 0x3FF, \quad PTI = (L \gg 12)\ \&\ 0x3FF, \quad d = L\ \&\ 0xFFF.$$

CR3 base 0x5000 and next PT base 0x6000 yield:

Level	Index	Entry address	Entry value	Meaning
Directory	0x048	0x5000+4×0x048=0x5120	0x00006007	Present, next base 0x6000, RW/user
Table	0x345	0x6000+4×0x345=0x6D14	0x00ABC007	Present, frame 0xABC, RW/user

Offset 0x678 gives PA0xABC000+0x678=0xABC678. This path uses directory, table, and target page, each 4 KiB. Table-allocation costs are separate from an individual walk.

IA-32 linear-address walk (continued)



An absent PDE stops before reading the PT; an absent PTE cannot supply a usable frame. On a CPU supporting 4 MiB pages, `CR4.PSE=1` permits PDE `PS=1` to terminate at the directory and use 22 offset bits. With `CR4.PSE=0`, 32-bit paging ignores PS and treats the PDE as a reference to a 4 KiB page table, not a 4 MiB mapping. This differs from the fixed trace.

Canonical x86-64 addresses and four levels

Fix 48-bit canonical virtual addresses, four levels,4 KiB pages,8-byte entries. Index fields are[47:39], [38:30],[29:21],[20:12], with offset[11:0]. Each table has 512 entries. First require 0≤v<2^64, then require every bit 63:48 to repeat bit 47.

$$canonical(v) \iff (v \gg 48) = \begin{cases} 65535, & ((v \gg 47)\ \&\ 1) = 1, \\ 0, & ((v \gg 47)\ \&\ 1) = 0. \end{cases}$$

Comparing only bit 63 to bit 47 is insufficient. v=0x0001_0000_0000_0000 has both 0 but upper 16 bits=1, so is noncanonical. Flipping any one sign-extension bit of a legal address invalidates it. Ranges 0…0x00007FFFFFFFFFFF and 0xFFFF800000000000…0xFFFFFFFFFFFFFFFF are legal; the interval between is illegal.

For VA 0x00007FFF_FFFFF123, bit 47=0 and upper 16 bits=0. CR3=0x1000; subsequent bases are 0x2000,0x3000,0x4000.

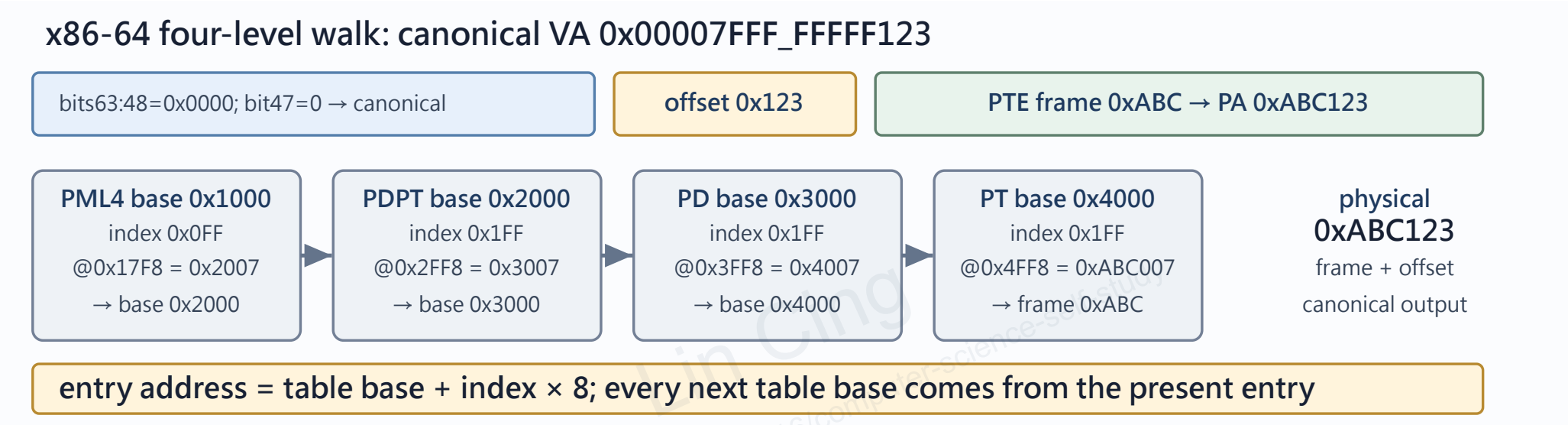
Canonical x86-64 addresses and four levels (continued)

Level	Index	Offset	Entry address	Entry value	Next base/frame
PML4	0x0FF	0x7F8	0x17F8	0x2007	PDPT 0x2000
PDPT	0x1FF	0xFF8	0x2FF8	0x3007	PD0x3000
PD	0x1FF	0xFF8	0x3FF8	0x4007	PT0x4000
PT	0x1FF	0xFF8	0x4FF8	0xABC007	Frame 0xABC

Every address is base+index×8; every next base comes from the present entry. Offset 0x123 is retained, so PA=0xABC123.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Canonical x86-64 addresses and four levels (continued)



2 MiB and 1 GiB encodings end higher in the walk and require other formats/alignment and larger offsets. The 9/9/9/9/12 split applies only to this 4 KiB configuration.

Mapping and operation permission

Use four-level 48-bit x86-64 paging, user-mode `CPL=3`, NX-capable hardware with `IA32_EFER.NXE=1`, and no intervening SMEP, SMAP, PKE or other added restriction. Canonical check precedes all table memory accesses. Present establishes usable entries, U/S restricts user access at every level, effective RW restricts writes, and NX restricts instruction fetch.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

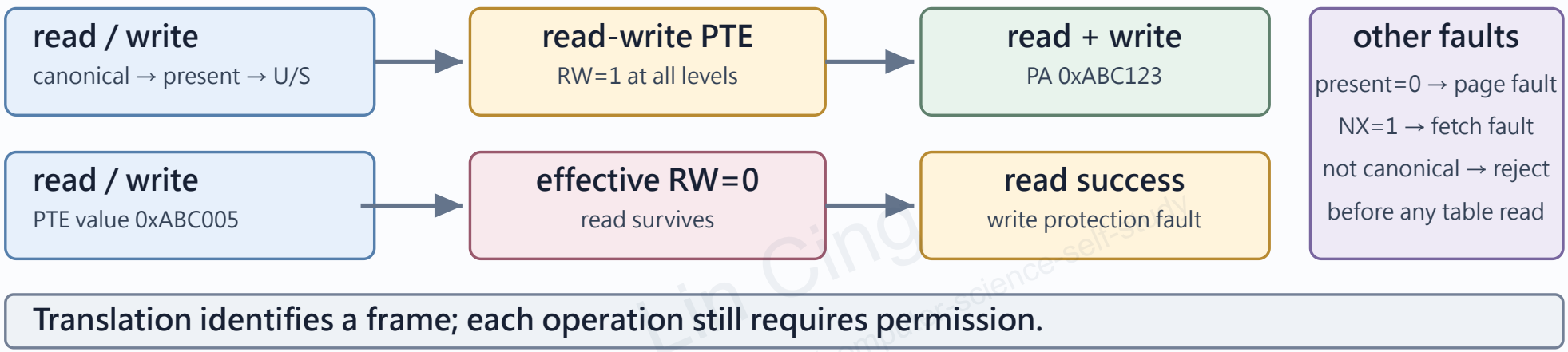
Mapping and operation permission (continued)

Case	Canonical/mapping	Permissions	Read	Write	Fetch
Normal trace	Canonical, four present levels	All user/RW, NX0	0xABC123	0xABC123	Allowed
PTE 0xABC005	Same present path	User, RW0, NX0	0xABC123	Protection fault	Allowed
PML4[0x0FF] absent	Canonical, no mapping	No effective mapping	Page fault	Page fault	Instruction page fault
PTE NX1	Canonical, present	User/RW, NX1	Allowed	Allowed	Fetch fault
VA 0x00008000_00000000	Noncanonical	No walk	Reject	Reject	Reject

https://github.com/xinhi6716/computer-science-self-study

Mapping and operation permission (continued)

x86-64 permission trace after canonical and present checks



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mapping and operation permission (continued)

The last address has bit 47=1 but upper 16 bits=0. The canonical address 0x00007FFF_FFFFF000 still page-faults if any required level is absent. The illegal interval ends at 0xFFFF7FFFFFFFFFFF, immediately below the upper canonical range. For a user write, effective RW=0 causes a protection fault; supervisor writes also depend on `CR0.WP` and are outside this table. If `EFER.NXE=0`, an entry with XD/NX set has a reserved bit and causes a reserved-bit page fault, rather than simply preventing fetch. A root, frame, and valid canonical input do not alone authorize every operation. PAE, five levels, large pages, or another physical width require the matching masks and entries. Updated entries need required TLB invalidation; a stale cached permission can outlive a changed PTE.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ARMv7-A short descriptors

Fix AArch32, short descriptors, TTBCR.N=0, TTBCR.PD0=0, SCTLR.AFE=0, SCTLR.TRE=0, PL1 table construction, and 4 KiB small pages. P1 and P2 use distinct, unreused ASIDs, e.g. 1 and 2. This is not AArch64 long-descriptor translation. N=0 selects TTBR0 for all input VAs. The complete L1 has 4096 four-byte entries,16 KiB, aligned 16 KiB. Sparse use does not permit allocating only the populated entries; other entries need valid architectural encodings, including invalid ones.

$$L1addr = TTBR0 + ((VA \gg 20)\ \&\ 0xFFF) \times 4.$$

A coarse L1 descriptor points to a256-entry L2,1 KiB, aligned 1 KiB; bits 31:10 supply its base.

$$L2addr = L2base + ((VA \gg 12)\ \&\ 0xFF) \times 4, \qquad PA = frame + (VA\ \&\ 0xFFF).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ARMv7-A short descriptors (continued)

Storage	Base	Inclusive range	Size/alignment
P1 root	0x8000	0x8000–0xBFFF	16 KiB, base mod 0x4000=0
P2 root	0xC000	0xC000–0xFFFF	16 KiB, base mod 0x4000=0
P1 L2	0x10000	0x10000–0x103FF	1 KiB, base mod 0x400=0
P2 L2	0x10400	0x10400–0x107FF	1 KiB, base mod 0x400=0

All four ranges are distinct. Only the final data frame is shared.

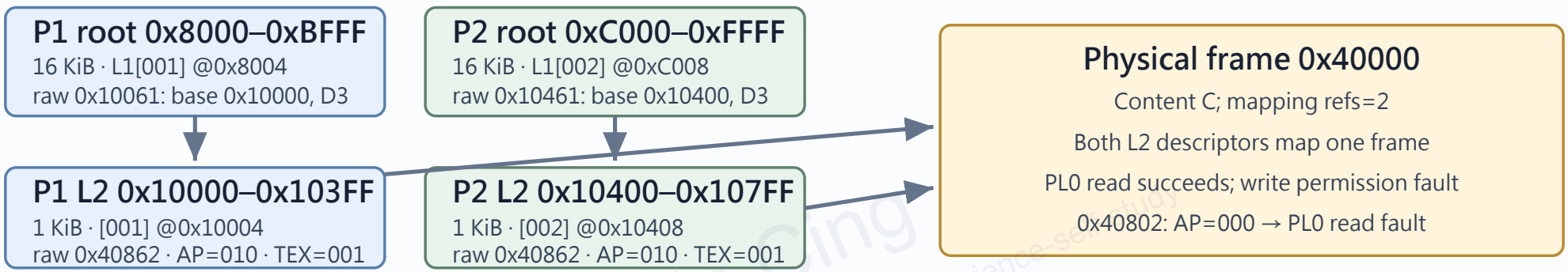
Process/VA	L1 index/address	L2 base	L2 index/address	Frame	PA
P1 0x00101000	0x001 /0x8004	0x10000	0x001 /0x10004	0x40000	0x40000
P2 0x00202000	0x002 /0xC008	0x10400	0x002 /0x10408	0x40000	0x40000
P1 0x00101A5C	0x001 /0x8004	0x10000	0x001 /0x10004	0x40000	0x40A5C

Offset[11:0]=0xA5C does not enter either table index. Keeping only 8bits would incorrectly give 0x4005C.

ARMv7-A short descriptors (continued)

ARMv7-A: separate translation tables, one shared read-only frame

TTBCR.N=0 · SCTLR.AFE=0/TRE=0 · DACR.D3=Client



L1: 4096×4 B=16 KiB, aligned 16 KiB. L2: 256×4 B=1 KiB, aligned 1 KiB.

Invalid: TTBR0=0x9000 misaligned; L2=0xA000/0xB000 overlaps L1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Raw ARM words, domains, AFE and AP

Both L1 descriptors use domain 3; DACR. D3=01 means Client. Domain is in L1 and inherited by L2. With AFE=0, small-page AP[2] is bit 9 and AP[1:0] bits 5:4, not section bits 11:10.

Descriptor	Raw	Decode
P1 L1	0x10061	Type 01 coarse, base 0x10000, domain 3
P2 L1	0x10461	Type 01 coarse, base 0x10400, domain 3
Both L2	0x40862	Frame 0x40000, type 10, XN0, nG1, AP010, TEX001, C=B=0

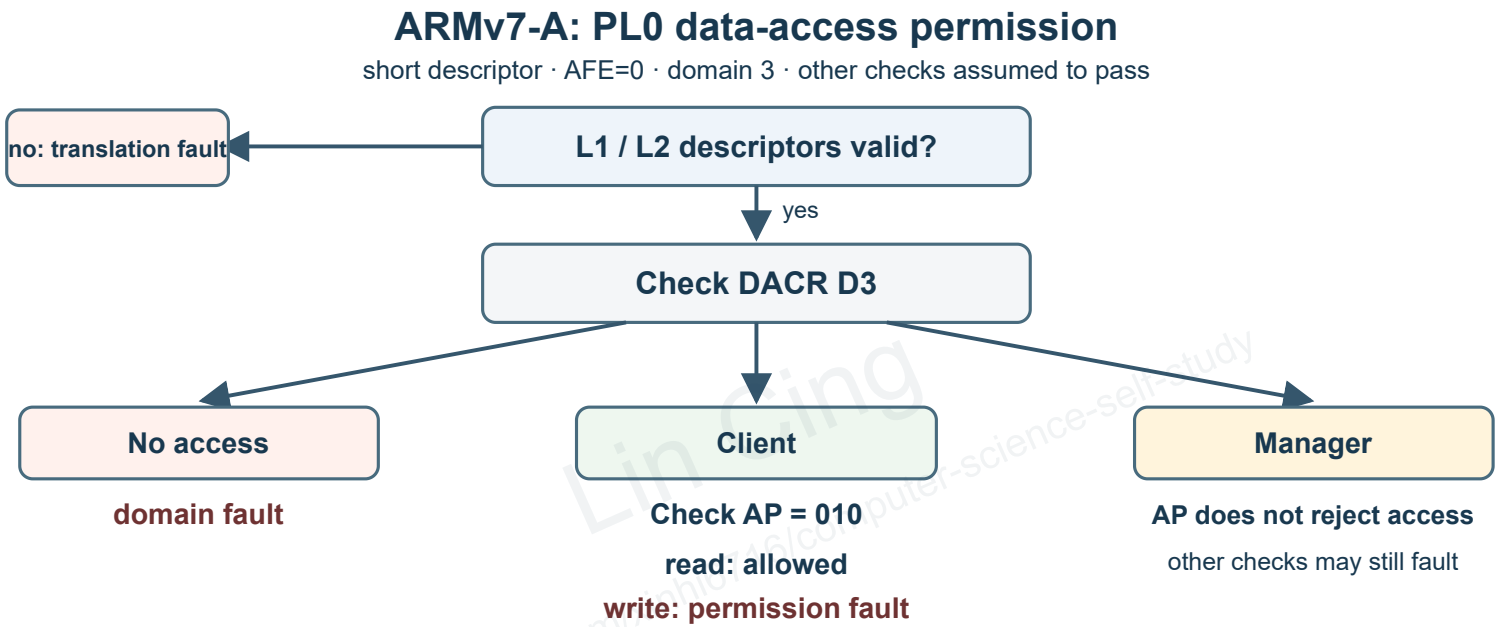
Decode field=(raw>>shift)&mask. AP2=(raw>>9)&1, AP1=(raw>>5)&1, AP0=(raw>>4)&1, TEX=(raw>>6)&7, C=(raw>>3)&1, B=(raw>>2)&1, XN=raw&1. AP010 gives PL1 read/write and PL0 read-only. With TRE0, TEX001/C0/B0 means Normal non-cacheable. Fetch additionally assumes WXN=UWXN=0, readable mapping, Client domain, and no other restriction. XN0 alone cannot prove executable Strongly-ordered memory.

Raw ARM words, domains, AFE and AP (continued)

0x40802 instead decodes AP000 and denies PL0 read/write even though the frame is valid. 0x40022 has AP010 but TEX/C/B=0/0/0, Strongly-ordered; it only illustrates data AP, not guaranteed fetch.

Mode	PL0 read	PL0 write	Reason
AFE0, Client, AP010	Success	Permission fault	Read-only PL0
AFE0, No access domain	Domain fault	Domain fault	Domain rejects first
AFE0, Manager domain 11	Not rejected by AP	Not rejected by AP	Manager bypasses AP; other faults remain possible
AFE1, Client, same 0x40862	Access-flag fault	Access-flag fault	AP0 is now access flag 0; software-managed flag model
AFE1, Client,0x40872	Success	Success	Flag 1, AP[2:1]=01 grants PL0 RW

Raw ARM words, domains, AFE and AP (continued)



A valid mapping is not enough: the domain and access permissions also matter.

Changing DACR or AFE changes the interpretation, so the AFE0 read-only result cannot be reused unchanged.

ARM unmap, mapping counts and stale translations

Initially the frame contains C and has exactly these two mappings, with no other references or pins; mapping count is 2, both entries 0x40862, Client parents, TTBCR.PD0=0, distinct unreused ASIDs 1 and 2 for P1 and P2, and no cached translation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ARM unmap, mapping counts and stale translations (continued)

State/operation	Translation/permission	Result	Content/references
S0 P1 read	0x00101000→0x40000, AP permits	Success	C, refs 2
S0 P1 write	Valid mapping, AP denies	Permission fault	C unchanged
S0 P2 read	0x00202000→0x40000, AP permits	Success	C, refs 2
S0 P2 write	Valid mapping, AP denies	Permission fault	C unchanged
S1 remove P1	L2 invalid, required TLB maintenance complete	P1 translation fault	P2 remains, refs 1
S2 P2 read	Independent tables still valid	Success	refs 1
S3 remove P2	L2 invalid, maintenance complete	P2 translation fault	refs 0; reclaim only after all stale translations are unusable

ARM unmap, mapping counts and stale translations (continued)

An absent mapping is not a valid-but-read-only mapping. After descriptor changes, perform architecture-specific VA/ASID TLBI and required barriers on every relevant core that might hold the old translation; wait for completion before reclaiming the frame. nG1 associates translations with their ASIDs; ASID does not determine domain or AP. Clearing a PTE alone cannot authorize frame reuse while an old TLB can still access it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Table storage must be valid before sharing is analyzed

Invalid layout	Failure
P2 TTBR0=0x9000, N0	0x9000 mod 0x4000=0x1000, root misaligned
P1 root 0x8000, P2 root 0x9000	Ranges 0x8000–BFFF and 9000–CFFF overlap
P1 L2=0xA000	Inside P1 root
P2 L2=0xB000	Inside P1 root
Only 4 KiB allocated for L1	4096×4 entries require 16 KiB

Legal sharing has independent roots/L2 storage and one shared frame. Verify page identity, storage, domain/AP/XN, references, and TLB state together. A writable P1 mapping would modify the frame P2 reads; that is writable sharing, not read-only code. Preserving P2's old data while P1 changes requires COW or a separate frame, not merely changing permissions in one entry.

Chapter conclusions

- A mapping and its permissions are distinct parts of access validation
- Classify fragmentation before comparing allocation schemes
- Effective access time is a model with explicit cache and fault assumptions

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 8. Intel 64 and IA-32 SDM, Vol. 3A; Arm Architecture Reference Manual ARMv7-A/R.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study